

JavaScript Project

 **Create a module that exports a function and import it into another file.**

Objective: This project demonstrates how to use JavaScript modules to import and export functions. Since OneCompiler does not support multiple files in Node.js, we will simulate module exports and imports within a single file.

Import and Use the Module

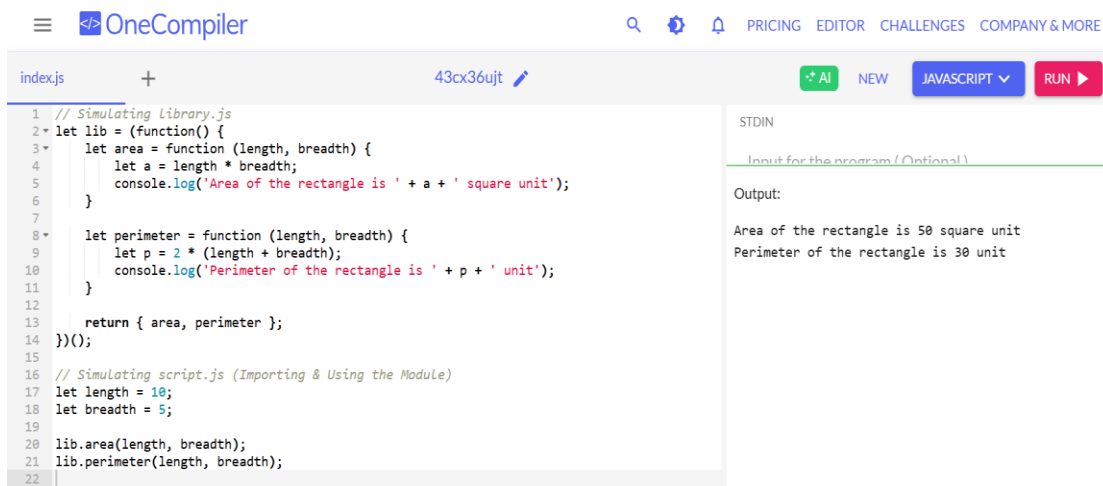
// Simulating script.js (Importing & Using the Module)

let length = 10;

let breadth = 5;

lib.area(length, breadth);

lib.perimeter(length, breadth);



The screenshot shows the OneCompiler web IDE. The editor displays a JavaScript file named 'index.js' with the following code:

```
1 // Simulating Library.js
2 let lib = (function() {
3   let area = function (length, breadth) {
4     let a = length * breadth;
5     console.log('Area of the rectangle is ' + a + ' square unit');
6   }
7
8   let perimeter = function (length, breadth) {
9     let p = 2 * (length + breadth);
10    console.log('Perimeter of the rectangle is ' + p + ' unit');
11  }
12
13  return { area, perimeter };
14 })();
15
16 // Simulating script.js (Importing & Using the Module)
17 let length = 10;
18 let breadth = 5;
19
20 lib.area(length, breadth);
21 lib.perimeter(length, breadth);
22
```

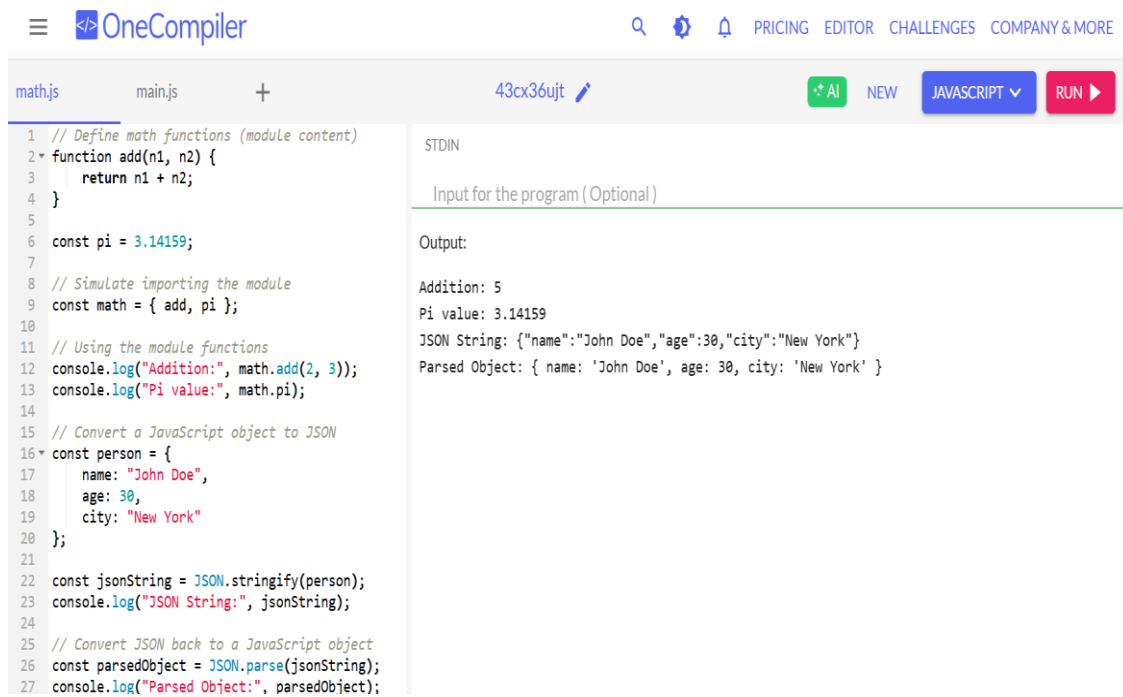
The right sidebar shows the 'Output' section with the following text:

```
Output:
Area of the rectangle is 50 square unit
Perimeter of the rectangle is 30 unit
```

Convert a JavaScript object to JSON and back using JSON.stringify() and JSON.parse().

Objective: This project covers two key JavaScript concepts:

1. Converting a JavaScript Object to JSON and Back using JSON.stringify() and JSON.parse().
2. Including a JavaScript File in Another JavaScript File using ES6 Modules, CommonJS, and script tags.



The screenshot shows the OneCompiler web interface. The editor contains the following JavaScript code:

```
1 // Define math functions (module content)
2 function add(n1, n2) {
3   return n1 + n2;
4 }
5
6 const pi = 3.14159;
7
8 // Simulate importing the module
9 const math = { add, pi };
10
11 // Using the module functions
12 console.log("Addition:", math.add(2, 3));
13 console.log("Pi value:", math.pi);
14
15 // Convert a JavaScript object to JSON
16 const person = {
17   name: "John Doe",
18   age: 30,
19   city: "New York"
20 };
21
22 const jsonString = JSON.stringify(person);
23 console.log("JSON String:", jsonString);
24
25 // Convert JSON back to a JavaScript object
26 const parsedObject = JSON.parse(jsonString);
27 console.log("Parsed Object:", parsedObject);
```

The output on the right is as follows:

```
STDIN
Input for the program (Optional)

Output:
Addition: 5
Pi value: 3.14159
JSON String: {"name":"John Doe","age":30,"city":"New York"}
Parsed Object: { name: 'John Doe', age: 30, city: 'New York' }
```

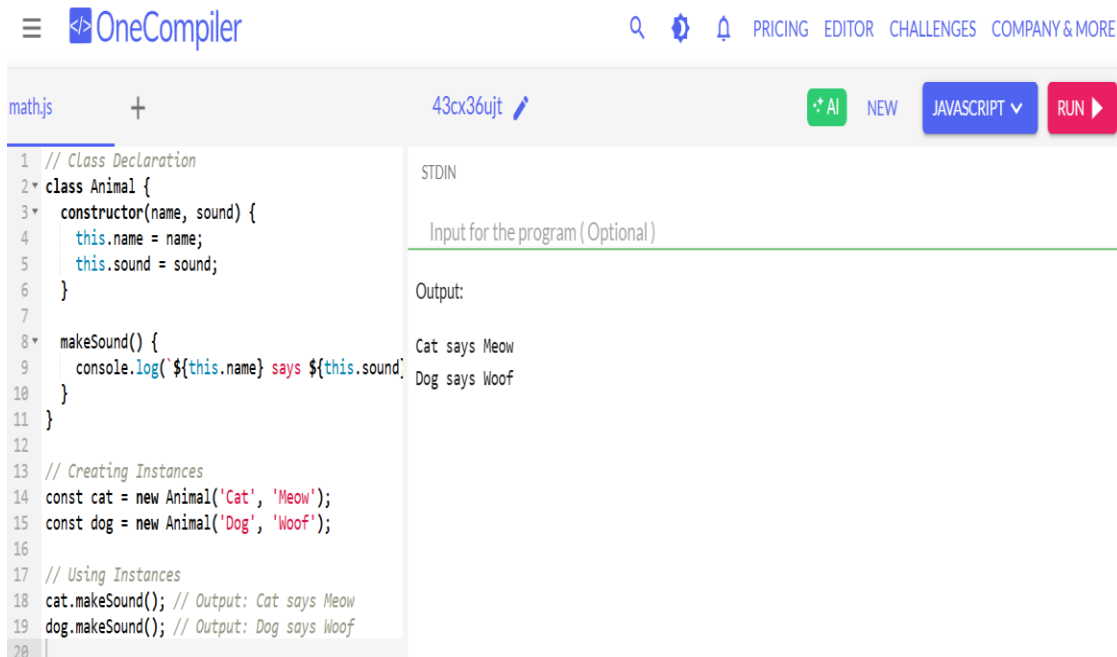
Conclusion summary:

- **Combines module and main file** since OneCompiler doesn't support multiple files.
- **Manually simulates module imports** using an object (math).
- **Ensures compatibility** by keeping everything in one script.

Define a class **Animal** with properties and a method, and create instances.

Objective: Create and use classes to define blueprints for creating objects with similar properties and behaviors.

Classes provide a way to implement object-oriented programming (OOP) concepts such as encapsulation, inheritance, and polymorphism.



The screenshot shows the OneCompiler web IDE. The editor has a file named `math.js` with the following JavaScript code:

```
1 // Class Declaration
2 class Animal {
3   constructor(name, sound) {
4     this.name = name;
5     this.sound = sound;
6   }
7
8   makeSound() {
9     console.log(`${this.name} says ${this.sound}`);
10  }
11 }
12
13 // Creating Instances
14 const cat = new Animal('Cat', 'Meow');
15 const dog = new Animal('Dog', 'Woof');
16
17 // Using Instances
18 cat.makeSound(); // Output: Cat says Meow
19 dog.makeSound(); // Output: Dog says Woof
20
```

The right-hand pane shows the execution results. Under "STDIN", there is a text input field with the placeholder "Input for the program (Optional)". Under "Output", the following text is displayed:

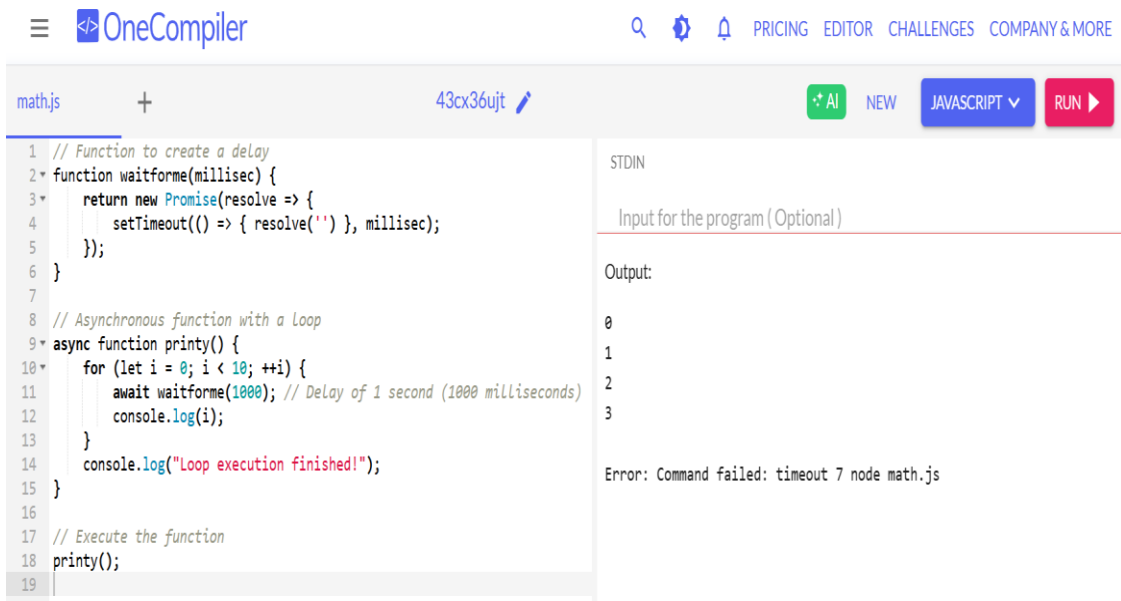
```
Cat says Meow
Dog says Woof
```

Working conclusion:

- One file execution (OneCompiler doesn't support multiple files easily).
- Uses ES6 class syntax (supported in OneCompiler's Node.js runtime).
- Simple OOP demonstration with constructors and methods.

 **Write a promise that resolves after a delay and convert it into an async function using async/await.**

Objective: The objective of this project is to demonstrate how to introduce a delay inside a loop in JavaScript using async/await with Promises. By implementing this, we can control the execution flow of asynchronous operations without blocking the main thread.



```
1 // Function to create a delay
2 function waitforme(millsec) {
3   return new Promise(resolve => {
4     setTimeout(() => { resolve('') }, millsec);
5   });
6 }
7
8 // Asynchronous function with a loop
9 async function printy() {
10   for (let i = 0; i < 10; ++i) {
11     await waitforme(1000); // Delay of 1 second (1000 milliseconds)
12     console.log(i);
13   }
14   console.log("Loop execution finished!");
15 }
16
17 // Execute the function
18 printy();
19
```

STDIN

Input for the program (Optional)

Output:

0
1
2
3

Error: Command failed: timeout 7 node math.js

Conclusion Summary: By implementing async/await with Promises in loops, JavaScript allows controlled execution of asynchronous tasks without blocking the main thread. This technique is especially helpful when working with timers, animations, and API calls that require controlled delays.